

THÉORIE DES GRAPHS

Séance 4 : Plus Court Chemin Algorithme de Dijkstra

L2 - Génie Informatique • Semestre 3

Ing. GBAGUIDI AISSE Alexandre
ESGC VERECHAGUINE A.K. • Cotonou, Bénin

Plan de la séance



Problème

Plus court chemin pondéré

10 min



Relaxation

Principe de mise à jour

10 min



Dijkstra

Algorithme glouton

25 min



File de priorité

Tas binaire, complexité

15 min



Preuve

Correction algorithme

10 min



Limitations

Poids négatifs

10 min



TD (1h)

4 exercices Dijkstra

Papier



TPE (1h30)

Implémenter Dijkstra

Code C++

Objectifs de la séance

1

Comprendre le problème du plus court chemin

Trouver le chemin de coût MINIMAL entre deux sommets
Graphe pondéré (chaque arête a un poids positif)
Différence avec BFS : BFS = arêtes poids 1

2

Maîtriser le principe de relaxation

Mettre à jour la distance estimée d'un sommet
Si nouveau chemin plus court → mettre à jour
Base de tous les algorithmes de plus court chemin

3

Implémenter l'algorithme de Dijkstra

Algorithme glouton avec file de priorité
Complexité $O((V+E) \log V)$ avec tas binaire
Retourne distance[] et parent[] pour reconstruction

Objectifs de la séance (suite)

4 Utiliser une file de priorité efficacement

Tas binaire (Min-Heap) pour extraire le minimum
Opérations `insert()` et `extractMin()` en $O(\log V)$
Impact majeur sur la complexité finale

5 Prouver la correction de Dijkstra

Invariant : sommets traités ont distance optimale
Démonstration par induction
Comprendre pourquoi Dijkstra est optimal

6 Identifier les limitations

Dijkstra ne fonctionne QUE si poids ≥ 0
Poids négatifs $\rightarrow$ Bellman-Ford nécessaire
Comprendre POURQUOI cette limitation existe

Rappel Séances 2 et 3 : BFS et DFS

Critère	BFS (Séance 2)	DFS (Séance 3)
Structure	File FIFO	Pile LIFO / Récursion
Exploration	Largeur (niveau par niveau)	Profondeur (au plus loin)
Plus court chemin	✓ Oui (NON pondéré)	✗ Non
Complexité	$O(V + E)$	$O(V + E)$
Applications	Distance, composantes, biparti	Cycle, tri topo, SCC



Limitation BFS/DFS : Ils ne gèrent PAS les graphes pondérés ! Pour le plus court chemin pondéré → Dijkstra.

PARTIE 2

Problème du Plus Court Chemin

"Comment aller de A à B au moindre coût ?"

Problème du plus court chemin



Définition

Étant donné un graphe $G = (V, E)$ pondéré où chaque arête (u, v) a un poids $w(u, v) \geq 0$, et un sommet source s , trouver le chemin de coût minimal de s vers tous les autres sommets.

Exemple simple

```
A --5--> B
|         |
2         1
|         |
v         v
C --3--> D
```

Plus court $A \rightarrow D$:

- $A \rightarrow B \rightarrow D = 5 + 1 = 6$
- $A \rightarrow C \rightarrow D = 2 + 3 = 5 \checkmark$

Notation

- $d[v]$ = distance minimale de s à v
- $\text{parent}[v]$ = prédécesseur de v dans le chemin optimal
- $w(u, v)$ = poids de l'arête $u \rightarrow v$

Objectif : calculer $d[v]$ pour tous les sommets v

Applications réelles du plus court chemin



Navigation GPS

- Itinéraire le plus rapide
- Poids = temps de trajet
- Google Maps, Waze
- Trafic en temps réel



Logistique

- Livraison au moindre coût
- Poids = distance/coût
- DHL, FedEx
- Optimisation de tournées



Routage réseau

- Paquets sur Internet
- Poids = latence/bande
- Protocole OSPF
- Mise à jour dynamique



Finance

- Arbitrage de devises
- Poids = taux de change
- Détecter opportunités
- Cycles négatifs = profit

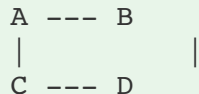


Dans tous ces cas, on cherche à minimiser un coût (temps, distance, argent). Dijkstra résout ce problème efficacement.

Pourquoi BFS ne suffit pas ?

△ Rappel : BFS trouve le plus court chemin en nombre d'arêtes (graphes non pondérés)

BFS - Graphe non pondéré



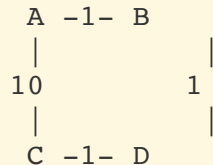
BFS depuis A vers D :

A→C→D = 2 arêtes

A→B→D = 2 arêtes

→ Tous les chemins = 2 arêtes ✓

Dijkstra - Graphe pondéré



Dijkstra depuis A vers D :

A→B→D = 1+1 = 2 ✓

A→C→D = 10+1 = 11

→ Plus court : A→B→D (coût 2)

✗ BFS ne distingue PAS les poids : il considère A→B→D et A→C→D identiques (2 arêtes chacun). Seul Dijkstra voit que A→B→D coûte 2 et A→C→D coûte 11.

Algorithmes de plus court chemin

Algorithme	Type	Poids négatifs ?	Complexité	Usage
BFS	Parcours	N/A (non pondéré)	$O(V+E)$	Graphes simpl
Dijkstra	Glouton	✗ Non	$O((V+E) \log V)$	Poids ≥ 0
Bellman-Ford	Prog. Dyn.	✓ Oui	$O(VE)$	Poids négatifs
Floyd-Warshall	Prog. Dyn.	✓ Oui	$O(V^3)$	Tous paires
A*	Heuristique	✗ Non	Variable	GPS (heuristiq



Focus Séance 4 : Nous étudions Dijkstra en détail. Bellman-Ford sera vu en Séance 6.

PARTIE 3

Principe de Relaxation

"Améliorer progressivement les estimations"

Qu'est-ce que la relaxation ?

⚡ Définition : Relaxation d'une arête

Relaxer une arête (u,v) signifie vérifier si passer par u permet d'améliorer la distance estimée vers v . Si $d[u] + w(u,v) < d[v]$, alors on met à jour $d[v] = d[u] + w(u,v)$ et $\text{parent}[v] = u$.

```
RELAX(u, v, w):
```

```
    // Vérifier si amélioration
    if d[v] > d[u] + w(u,v):

        // Mettre à jour distance
        d[v] = d[u] + w(u,v)

        // Mettre à jour parent
        parent[v] = u
```

Visualisation

u ----5----> v

$d[u] = 3$
 $d[v] = 10$ (actuel)

Relaxer (u,v) :
 $3 + 5 = 8 < 10$ ✓
→ $d[v] = 8$
→ $\text{parent}[v] = u$

💡 La relaxation est le cœur de TOUS les algorithmes de plus court chemin (Dijkstra, Bellman-Ford, etc.)

Exemple visuel de relaxation

Graphe avec distances actuelles : s (source) a $d[s] = 0$, tous les autres ont $d[v] = \infty$

Étape 1 : Relaxer arête $s \rightarrow a$ (poids 4)

Avant : $d[a] = \infty$ | Calcul : $d[s] + 4 = 0 + 4 = 4 < \infty$ ✓ | Après : $d[a] = 4$, $\text{parent}[a] = s$

Étape 2 : Relaxer arête $s \rightarrow b$ (poids 2)

Avant : $d[b] = \infty$ | Calcul : $d[s] + 2 = 0 + 2 = 2 < \infty$ ✓ | Après : $d[b] = 2$, $\text{parent}[b] = s$

Étape 3 : Relaxer arête $b \rightarrow a$ (poids 1)

Avant : $d[a] = 4$ | Calcul : $d[b] + 1 = 2 + 1 = 3 < 4$ ✓ | Après : $d[a] = 3$, $\text{parent}[a] = b$ (AMÉLIORATION !)

✓ À chaque relaxation réussie, on se rapproche de la distance optimale. Dijkstra relaxe systématiquement toutes les arêtes.

Propriété de la relaxation

1

Borne supérieure

$d[v]$ est toujours une BORNE SUPÉRIEURE de la distance réelle.

Après initialisation : $d[s] = 0$, $d[v] = \infty$ pour $v \neq s$

Après chaque relaxation : $d[v] \geq \text{distance_réelle}(s, v)$

2

Convergence

Une fois $d[v]$ atteint la distance optimale, il ne change plus.

Si on relaxe suffisamment, $d[v]$ converge vers la vraie distance.

Dijkstra garantit cette convergence en $O((V+E) \log V)$

3

Chemin optimal

Si $d[v] = \text{distance_optimale}(s, v)$, alors $\text{parent}[]$ encode un chemin optimal.

On peut reconstruire le chemin en remontant : $v \rightarrow \text{parent}[v] \rightarrow \text{parent}[\text{parent}[v]] \rightarrow \dots \rightarrow s$

Pourquoi la relaxation fonctionne ?



Intuition

Imaginez que vous cherchez le chemin le moins cher dans une ville. Au début, vous ne connaissez que les routes directes depuis votre point de départ. À chaque fois que vous trouvez un nouveau carrefour, vous vérifiez si passer par ce carrefour permet d'atteindre d'autres destinations moins cher. La relaxation fait exactement ça !



Inégalité triangulaire

Pour tout chemin $s \rightarrow u \rightarrow v$:

$$\text{distance}(s,v) \leq \text{distance}(s,u) + w(u,v)$$

C'est cette propriété qui justifie la relaxation :

Si $d[u] + w(u,v) < d[v]$
→ On a trouvé un meilleur chemin !



Garantie mathématique

Si on relaxe toutes les arêtes dans le bon ordre, on obtient les distances optimales.

Dijkstra choisit l'ordre intelligemment :
→ Traiter d'abord les sommets proches
→ File de priorité pour efficacité

Résultat : distances optimales
garanties en $O((V+E) \log V)$!



Important : La relaxation seule ne suffit pas. Il faut aussi le BON ORDRE de traitement → C'est là que Dijkstra intervient !

PARTIE 4

Algorithme de Dijkstra

"L'algorithme glouton optimal"

Edsger W. Dijkstra (1930-2002)



Informaticien néerlandais

- Né à Rotterdam, Pays-Bas (1930)
- Pionnier de l'informatique théorique
- Prix Turing 1972 ("Nobel de l'informatique")

🎯 L'algorithme de Dijkstra

- Inventé en 1956, publié en 1959
- Trouvé en 20 minutes dans un café !
- Motivation : calculer le plus court chemin entre Rotterdam et Groningen

📖 Autres contributions

- Programmation structurée
- Problème des philosophes dîneurs
- Algorithme du banquier
- Citation célèbre : "Go To Statement Considered Harmful"



Impact aujourd'hui

L'algorithme de Dijkstra est utilisé dans :

- GPS et navigation
- Routage Internet (OSPF)
- Jeux vidéo (pathfinding)
- Robotique
- Réseaux télécoms
- Optimisation logistique

Un des algorithmes les plus utilisés au monde !

Idée générale de l'algorithme



Idée centrale : Algorithme glouton (greedy)

À chaque étape, choisir le sommet NON VISITÉ qui a la plus petite distance estimée, puis relaxer toutes ses arêtes sortantes. Répéter jusqu'à ce que tous les sommets soient visités.

1. Initialisation

$d[s] = 0$, $d[v] = \infty$ pour $v \neq s$
 $\text{Parent}[v] = \text{null}$
File de priorité Q avec tous les sommets

2. Extraction

Extraire u de Q avec $d[u]$ minimal
Marquer u comme visité
 u a maintenant sa distance OPTIMALE

3. Relaxation

Pour chaque voisin v de u non visité :
Si $d[u] + w(u,v) < d[v]$:
 $\rightarrow d[v] = d[u] + w(u,v)$, $\text{parent}[v] = u$

4. Répétition

Répéter étapes 2-3 jusqu'à ce que Q soit vide
Résultat : $d[]$ contient les distances optimales

Structures de données nécessaires

Tableau distance[] (d[])

- Taille : V (nombre de sommets)
- $d[v]$ = distance minimale actuelle de s à v
- Initialisation : $d[s] = 0$, $d[v] = \infty$ (ou INT_MAX)
- Mise à jour : lors de chaque relaxation réussie

```
vector<int> distance(V, INT_MAX);
```

Tableau parent[]

- Taille : V
- $\text{parent}[v]$ = prédécesseur de v dans le chemin optimal
- Initialisation : $\text{parent}[v] = -1$ (ou null)
- Permet de reconstruire le chemin à la fin

```
vector<int> parent(V, -1);
```

File de priorité Q (Min-Heap)

- Contient les sommets non encore visités
- Priorité = distance actuelle $d[v]$
- Opérations : insert $O(\log V)$, extractMin $O(\log V)$
- Clé de l'efficacité de Dijkstra !

```
priority_queue<pair<int,int>> Q;
```

Algorithme de Dijkstra - Pseudo-code

DIJKSTRA(Graph G, vertex s):

```
// 1. Initialisation
for each vertex v in V:
    distance[v] =  $\infty$ 
    parent[v] = null
    Q.insert(v) // File de priorité

distance[s] = 0

// 2. Boucle principale
while Q is not empty:

    // Extraire sommet avec distance minimale
    u = Q.extractMin()

    // Relaxer toutes les arêtes sortantes de u
    for each neighbor v of u:

        // Si amélioration trouvée
        if distance[v] > distance[u] + w(u,v):
            distance[v] = distance[u] + w(u,v)
            parent[v] = u
            Q.decreaseKey(v) // Mettre à jour priorité

return distance[], parent[]
```



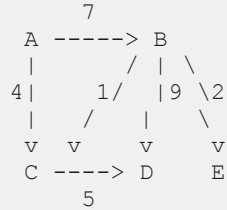
Points clés

- Init : $d[s]=0$, autres= ∞
- Q = file priorité (Min-Heap)
- extractMin() : $O(\log V)$
- Boucle : V fois
- Pour chaque arête : relaxation
- decreaseKey() si amélioration
- Complexité totale : $O((V+E) \log V)$

Exemple : Exécution de Dijkstra

Graphe pondéré orienté, source $s = A$. Objectif : calculer distances optimales vers tous les sommets.

Graphe G :



Arêtes avec poids :

- $A \rightarrow B$ (7), $A \rightarrow C$ (4)
- $B \rightarrow C$ (1), $B \rightarrow D$ (9), $B \rightarrow E$ (2)
- $C \rightarrow D$ (5)

État initial :

```
d[A] = 0 (source)
d[B] = ∞
d[C] = ∞
d[D] = ∞
d[E] = ∞
```

```
parent[] = tous null
```

```
Q = {A, B, C, D, E}
(file de priorité)
```

Exemple : Étapes 1-2

Étape 1 : Extraire A ($d[A] = 0$, minimum dans Q)

Relaxer voisins de A :

- $A \rightarrow B$ (poids 7) : $d[B] = \infty \rightarrow d[B] = 0 + 7 = 7 \checkmark$, $\text{parent}[B] = A$
- $A \rightarrow C$ (poids 4) : $d[C] = \infty \rightarrow d[C] = 0 + 4 = 4 \checkmark$, $\text{parent}[C] = A$

État après étape 1 : $d[A]=0$, $d[B]=7$, $d[C]=4$, $d[D]=\infty$, $d[E]=\infty$

$Q = \{B, C, D, E\}$ (A retiré)

Étape 2 : Extraire C ($d[C] = 4$, minimum dans Q)

Relaxer voisins de C :

- $C \rightarrow D$ (poids 5) : $d[D] = \infty \rightarrow d[D] = 4 + 5 = 9 \checkmark$, $\text{parent}[D] = C$

C n'a qu'un seul voisin D.

État après étape 2 : $d[A]=0$, $d[B]=7$, $d[C]=4$, $d[D]=9$, $d[E]=\infty$

$Q = \{B, D, E\}$ (C retiré)

Exemple : Étapes 3-4

Étape 3 : Extraire B ($d[B] = 7$, minimum dans Q)

Relaxer voisins de B :

- $B \rightarrow C$ (poids 1) : $d[C] = 4$, mais $7+1 = 8 > 4$ ✗ (pas d'amélioration)
- $B \rightarrow D$ (poids 9) : $d[D] = 9$, mais $7+9 = 16 > 9$ ✗ (pas d'amélioration)
- $B \rightarrow E$ (poids 2) : $d[E] = \infty \rightarrow d[E] = 7 + 2 = 9$ ✓, $\text{parent}[E] = B$

État après étape 3 : $d[A]=0$, $d[B]=7$, $d[C]=4$, $d[D]=9$, $d[E]=9$
 $Q = \{D, E\}$ (B retiré)

Étapes 4-5 : Extraire D puis E

Étape 4 : Extraire D ($d[D] = 9$)

- D n'a pas de voisins sortants $\rightarrow$ aucune relaxation
- $Q = \{E\}$

Étape 5 : Extraire E ($d[E] = 9$)

- E n'a pas de voisins sortants $\rightarrow$ aucune relaxation
- $Q = \{\}$ (vide)

Algorithme terminé !

Résultat final et reconstruction des chemins

✓ Distances optimales

Depuis A vers :

- A : distance = 0
- B : distance = 7
- C : distance = 4
- D : distance = 9
- E : distance = 9

Tous les sommets ont
leur distance OPTIMALE !

🔗 Tableau parent[]

parent[A] = null (source)
parent[B] = A
parent[C] = A
parent[D] = C
parent[E] = B

Permet de reconstruire
les chemins optimaux !

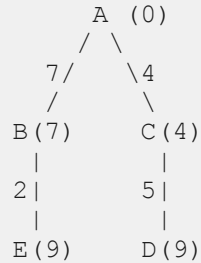
📌 Reconstruction des chemins

Chemin A→D : $D \leftarrow \text{parent}[D]=C \leftarrow \text{parent}[C]=A \rightarrow$ Chemin : A → C → D (coût 9)

Chemin A→E : $E \leftarrow \text{parent}[E]=B \leftarrow \text{parent}[B]=A \rightarrow$ Chemin : A → B → E (coût 9)

Visualisation : Arbre des plus courts chemins

Arbre des plus courts chemins depuis A :



Légende :

- Sommet (distance depuis A)
- Arêtes en gras = chemin optimal
- Toutes les distances sont OPTIMALES
- Complexité : $O((V+E) \log V)$

✅ Dijkstra garantit les distances optimales pour TOUS les sommets atteignables depuis la source !

PARTIE 5

File de Priorité et Tas Binaire

"La clé de l'efficacité de Dijkstra"

Pourquoi une file de priorité ?

? Problème à résoudre

À chaque itération de Dijkstra, on doit trouver le sommet NON VISITÉ avec la plus petite distance. Comment faire ça efficacement ?

Approche	Opération	Complexité par itération
✗ Recherche linéaire	Parcourir tous les sommets	$O(V)$
✓ File de priorité (tas)	extractMin() avec tas	$O(\log V)$

💡 Exemple concret

Graphe avec $V = 10\,000$ sommets, $E = 50\,000$ arêtes :

- Recherche linéaire : $O(10\,000^2) = O(100\,000\,000)$ opérations → 100 millions !
- File de priorité : $O((10\,000 + 50\,000) \log 10\,000) = O(60\,000 \times 14) = O(840\,000)$ → 840 mille
→ Gain de performance : ×119 fois plus rapide !

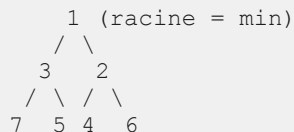
Tas binaire (Min-Heap)



Définition : Tas binaire minimum (Min-Heap)

Arbre binaire COMPLET où chaque nœud est plus PETIT que ses enfants. La racine contient toujours l'élément minimum.

Exemple de Min-Heap :



Propriété Min-Heap :
parent $\leq$ enfants

- $1 \leq 3$ et $1 \leq 2$ ✓
- $3 \leq 7$ et $3 \leq 5$ ✓
- $2 \leq 4$ et $2 \leq 6$ ✓



Propriétés importantes

- Arbre binaire COMPLET
(tous les niveaux remplis sauf dernier)
- Hauteur = $O(\log V)$
- Implémentation : tableau
 - Parent de i : $(i-1)/2$
 - Enfant gauche : $2i+1$
 - Enfant droit : $2i+2$
- Extraction du min : $O(\log V)$
- Insertion : $O(\log V)$

Opérations sur le tas binaire

1. insert(x) — Insertion

- Ajouter x à la fin du tableau
- Remonter x (bubble-up) tant que $\text{parent}(x) > x$
- Complexité : $O(\log V)$
- Utilisé pour ajouter sommets à Q

```
heap.push_back(x)
while parent(x) > x:
    swap(x, parent(x))
```

2. extractMin() — Extraction du minimum

- Retourner la racine (minimum)
- Mettre le dernier élément à la racine
- Descendre (bubble-down) pour rétablir propriété
- Complexité : $O(\log V)$

```
min = heap[0]
heap[0] = heap.back()
heap.pop_back()
bubble_down(heap[0])
```

3. decreaseKey(v, new_dist) — Diminuer clé

- Mettre à jour $\text{distance}[v] = \text{new_dist}$
- Remonter v (bubble-up) car priorité diminuée
- Complexité : $O(\log V)$
- Utilisé après relaxation réussie

```
distance[v] = new_dist
while parent(v) > v:
    swap(v, parent(v))
```

Implementation en C++

```
#include <queue>
#include <vector>
using namespace std;

// Dijkstra avec priority_queue de C++ STL
vector<int> dijkstra(Graph& g, int source) {
    int V = g.getNumVertices();
    vector<int> distance(V, INT_MAX);
    vector<bool> visited(V, false);

    // Min-Heap : pair<distance, vertex>
    // priority_queue est Max-Heap par défaut, donc inverser avec greater<>
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> Q;

    distance[source] = 0;
    Q.push({0, source}); // {distance, vertex}

    while (!Q.empty()) {
        // Extraire sommet avec distance minimale
        int u = Q.top().second;
        Q.pop();

        if (visited[u]) continue; // Déjà traité
        visited[u] = true;

        // Relaxer voisins
        for (auto edge : g.getNeighbors(u)) {
            int v = edge.dest;
            int weight = edge.weight;

            if (!visited[v] && distance[u] + weight < distance[v]) {
                distance[v] = distance[u] + weight;
                Q.push({distance[v], v}); // Insertion (pas de decreaseKey)
            }
        }
    }
    return distance;
}
```



En C++, on utilise `priority_queue` avec `greater<>` pour obtenir un Min-Heap. On insère plusieurs fois le même sommet plutôt que `decreaseKey`.

Impact sur la complexité de Dijkstra



Analyse détaillée avec file de priorité

- Initialisation : $O(V)$ — créer `distance[]`, `parent[]`, `Q`
- Boucle principale : V itérations
 - `extractMin()` : $O(\log V)$ par itération → Total : $O(V \log V)$
- Pour chaque arête (u,v) , on peut faire :
 - Relaxation : $O(1)$
 - `decreaseKey()` ou `insert()` : $O(\log V)$
 - Au total : E arêtes relaxées → $O(E \log V)$



Complexité finale : $O((V + E) \log V)$

Implémentation	<code>extractMin()</code>	<code>decreaseKey()</code>	Complexité totale
Recherche linéaire	$O(V)$	$O(1)$	$O(V^2)$
Tas binaire (Min-Heap)	$O(\log V)$	$O(\log V)$	$O((V+E) \log V)$
Tas de Fibonacci	$O(\log V)$	$O(1)$ amorti	$O(V \log V + E)$

PARTIE 6

Analyse de Complexité

"Pourquoi $O((V+E) \log V)$?"

Complexité détaillée de Dijkstra

1. Initialisation

- Créer $\text{distance}[V] = \{\infty, \dots, \infty\}$
- Créer $\text{parent}[V] = \{\text{null}, \dots\}$
- Créer file de priorité Q
- Insérer tous les V sommets

$O(V)$

2. Boucle principale

- V itérations (un par sommet)
- Chaque itération : $\text{extractMin}()$
- $\text{extractMin}()$ coûte $O(\log V)$

$O(V \log V)$

3. Relaxations

- Chaque arête relaxée au plus 1 fois
- E arêtes au total
- Chaque relaxation : $O(1) + \text{decreaseKey } O(\log V)$

$O(E \log V)$



COMPLEXITÉ TOTALE : $O(V) + O(V \log V) + O(E \log V) = O((V + E) \log V)$

Complexité selon l'implémentation

Implémentation	Structure	extractMin()	decreaseKey()	Complexité totale	Usage
Tableau simple	array	$O(V)$	$O(1)$	$O(V^2)$	Graphes denses
Tas binaire	binary heap	$O(\log V)$	$O(\log V)$	$O((V+E) \log V)$	Usage général ✓
Tas de Fibonacci	Fibonacci heap	$O(\log V)$	$O(1)$ amorti	$O(V \log V + E)$	Théorique
Tableau de listes	bucket queue	$O(1)$ amorti	$O(1)$	$O(V + E)$	Poids entiers petits

💡 Graphes denses ($E \approx V^2$) : Tableau simple : $O(V^2)$ acceptable car $E \approx V^2$

💡 Graphes épars ($E \ll V^2$) : Tas binaire : $O((V+E) \log V) \ll O(V^2) \rightarrow$ Bien meilleur !

Comparaison BFS vs Dijkstra

Critère	BFS	Dijkstra
Type de graphe	Non pondéré (ou poids = 1)	Pondéré (poids ≥ 0)
Structure	File FIFO	File de priorité (Min-Heap)
Ordre de visite	Niveau par niveau	Distance croissante
Complexité temps	$O(V + E)$	$O((V+E) \log V)$
Complexité espace	$O(V)$	$O(V)$
Plus court chemin	✓ Oui (nombre d'arêtes)	✓ Oui (somme des poids)
Poids négatifs	N/A	✗ Non supporté
Implémentation	Simple	Plus complexe (tas)

PARTIE 7

Preuve de Correction

"Pourquoi Dijkstra est-il optimal ?"

Invariant de l'algorithme



Invariant de boucle

À chaque itération, tous les sommets DÉJÀ EXTRAITS de Q ont leur distance OPTIMALE.
Une fois qu'un sommet u est extrait, $d[u]$ ne changera plus JAMAIS et égale la vraie distance minimale de s à u .

Sommets extraits (S)

- $d[u]$ = distance optimale ✓
- Plus de modifications possibles
- Dijkstra garantit l'optimalité

Sommets dans Q

- $d[v]$ = borne supérieure
- Peut encore être amélioré
- Sera optimal quand extrait



Propriété clé : Choix glouton optimal

Le sommet u extrait à chaque itération (celui avec $d[u]$ minimal dans Q) a DÉJÀ sa distance optimale.

Pourquoi ? Tout chemin alternatif vers u devrait passer par un autre sommet v encore dans Q , mais $d[v] \geq d[u]$, donc ce chemin serait au moins aussi long (car poids ≥ 0) $\rightarrow d[u]$ est optimal !

Démonstration (sketch)

1

Initialisation

Base : Après init, seul s est extrait avec $d[s] = 0$ = distance optimale ✓

2

Hypothèse d'induction

Supposons que tous les sommets extraits jusqu'ici ont distance optimale

3

Extraction de u

Soit u le prochain sommet extrait ($d[u]$ minimal dans Q). Montrons que $d[u]$ est optimal.

4

Raisonnement par l'absurde

Supposons $\exists$ un chemin plus court P vers u . Soit v le premier sommet de P encore dans Q . Alors $\text{distance}(s,v) < d[u]$, mais v est dans $Q \rightarrow$ contradiction car u a d minimal dans Q !



Conclusion : Par induction, tous les sommets extraits ont leur distance optimale $\rightarrow$ Dijkstra est CORRECT !

Pourquoi Dijkstra est optimal ?



1. Choix glouton toujours correct

Le choix du sommet avec distance minimale est TOUJOURS correct grâce à l'hypothèse poids ≥ 0 . Aucun chemin futur ne peut améliorer un sommet déjà extrait.



2. Relaxation exhaustive

Toutes les arêtes sont relaxées au moment opportun (quand le sommet source est extrait). Garantit qu'aucune amélioration n'est manquée.



3. Inégalité triangulaire respectée

Avec poids ≥ 0 , l'inégalité triangulaire garantit que passer par des sommets intermédiaires ne peut que rallonger ou égaler le chemin direct.



CONDITION CRITIQUE : Poids ≥ 0

Si poids négatifs $\rightarrow$ le choix glouton échoue ! Un sommet extrait pourrait être amélioré plus tard. $\rightarrow$ Utiliser Bellman-Ford.

PARTIE 8

Limitations et Poids Négatifs

"Quand Dijkstra échoue"

Le problème des poids négatifs

 **HYPOTHÈSE CRITIQUE DE DIJKSTRA : Tous les poids doivent être ≥ 0**

 **Pourquoi cette limitation ?**

Le choix glouton de Dijkstra repose sur le fait qu'une fois un sommet extrait, sa distance est FINALE.

Avec poids négatifs, un sommet déjà extrait pourrait être amélioré PLUS TARD via un chemin passant par une arête négative.

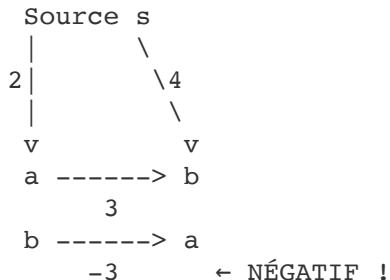
→ L'invariant "sommets extraits ont distance optimale" est BRISÉ !

 **Deux cas problématiques avec poids négatifs**

1. Arêtes négatives (mais pas de cycle négatif) → Dijkstra peut donner résultat INCORRECT
2. Cycle négatif (somme poids < 0) → Plus court chemin N'EXISTE PAS (on peut boucler infiniment)

Exemple : Dijkstra échoue avec poids négatifs

Graphe avec arête négative :



Arêtes :

$s \rightarrow a$ (2), $s \rightarrow b$ (4)

$a \rightarrow c$ (3), $b \rightarrow a$ (-3)

✗ Dijkstra calcule :

Étape 1 : Extraire s ($d[s]=0$)

Relaxer : $d[a]=2$, $d[b]=4$

Étape 2 : Extraire a ($d[a]=2$)

a est marqué VISITÉ ✓

Relaxer $a \rightarrow c$: $d[c]=2+3=5$

Étape 3 : Extraire b ($d[b]=4$)

Relaxer $b \rightarrow a$: $4+(-3)=1 < 2$

Mais a DÉJÀ VISITÉ → IGNORÉ !

Résultat Dijkstra : $d[a] = 2$

~~Vraie distance : $s \rightarrow b \rightarrow a = 1$~~



Pourquoi l'erreur ?

Dijkstra extrait a avec $d[a]=2$ et le marque comme visité car c'est le minimum dans Q.

Ensuite, quand b est extrait et qu'on découvre le chemin $s \rightarrow b \rightarrow a = 4+(-3) = 1 < 2$,

il est trop tard : a est déjà visité et ne sera plus mis à jour.

→ L'arête négative $b \rightarrow a$ permet d'améliorer un sommet DÉJÀ EXTRAIT, ce que Dijkstra interdit.

Solution : Algorithme de Bellman-Ford

✓ Bellman-Ford : L'alternative pour poids négatifs

Algorithme qui FONCTIONNE avec poids négatifs (sauf cycles négatifs).

Principe : Relaxer TOUTES les arêtes $V-1$ fois.

Complexité : $O(VE)$ — plus lent que Dijkstra mais universel.

Critère	Dijkstra	Bellman-Ford
Poids négatifs	✗ Non	✓ Oui
Détection cycle négatif	✗ Non	✓ Oui
Complexité	$O((V+E) \log V)$	$O(VE)$
Approche	Glouton (greedy)	Programmation dynamique
Usage	Poids ≥ 0 uniquement	Poids quelconques

📖 Bellman-Ford sera étudié en détail en Séance 6. Retenez : Dijkstra = rapide mais poids ≥ 0 , Bellman-Ford = lent mais universel.

PARTIE 9

Récapitulatif

"Ce qu'il faut retenir sur Dijkstra"

Points clés à retenir

1

Dijkstra = Plus court chemin avec poids ≥ 0

Trouve les distances minimales depuis une source vers tous les sommets. Hypothèse CRITIQUE : poids ≥ 0 .

2

Relaxation = cœur de l'algorithme

Améliorer progressivement les estimations : si $d[u] + w(u,v) < d[v] \rightarrow$ mettre à jour $d[v]$ et $parent[v]$.

3

File de priorité = efficacité

Tas binaire (Min-Heap) extrait le sommet avec distance minimale en $O(\log V)$. Clé de la complexité $O((V+E) \log V)$.

4

Algorithme glouton optimal

Le choix du sommet avec $d[u]$ minimal est toujours correct. Une fois extrait, $d[u]$ est FINAL et optimal.

5

Limitation : poids négatifs

Dijkstra ÉCHOUE avec poids < 0 . Utiliser Bellman-Ford pour poids négatifs ($O(VE)$, plus lent).

Quand utiliser Dijkstra ?

Situation	Algorithme à utiliser
Graphe non pondéré ?	BFS (plus simple, $O(V+E)$)
Poids tous ≥ 0 ?	✅ DIJKSTRA (optimal, $O((V+E) \log V)$)
Arêtes de poids négatifs ?	Bellman-Ford ($O(VE)$)
Cycle négatif possible ?	Bellman-Ford avec détection
Graphe très dense ($E \approx V^2$) ?	Dijkstra avec tableau simple ($O(V^2)$)
Besoin de tous-vers-tous ?	Floyd-Warshall ($O(V^3)$)



Règle d'or : Si poids $\geq 0 \rightarrow$ Dijkstra est le meilleur choix (rapide et optimal). Sinon $\rightarrow$ Bellman-Ford.

Tableau comparatif : BFS, DFS, Dijkstra

Critère	BFS	DFS	Dijkstra
Type graphe	Non pondéré	Non pondéré	Pondéré (≥ 0)
Structure	File FIFO	Pile LIFO	File priorité
Ordre visite	Niveau	Profondeur	Distance croissante
Plus court chemin	✓ (arêtes)	✗	✓ (poids)
Détection cycle	Possible	✓	Non
Tri topologique	✗	✓	✗
Complexité	$O(V+E)$	$O(V+E)$	$O((V+E) \log V)$
Espace	$O(V)$	$O(V)$	$O(V)$
Implémentation	Simple	Simple	Complexe (tas)